

ONLINE FOOD ORDERING SYSTEM

Final Examination (Output-Based)

Diploma in Information Technology (DIT)

Members:

Arboladora, Jhaneya N.
Gagap, Matthew Anthony B.
Marasigan, Flance Eleu John M.
Piol, Mirexelle Vincent C.
Toledo, Ivan P.

Submitted to:

Prof. Marc Anthony S. Bautista

Diploma in Information Technology 2-6

June 2026

I. DEVELOPMENT TEAM

Piol, Mirexelle Vincent C.

Back-end Developer

Toledo, Ivan P.

Front-end Developer

Gagap, Matthew Anthony B.

Paper

Arboladora, Jhaneya N.

Quality Assurance/Paper

Marasigan, Fliance Eleu John M.

Database Integration

II. INTRODUCTION

An enterprise-grade, object-oriented Graphical User Interface (GUI) application simulating a modern food delivery and ordering platform (e.g., GrabFood, Foodpanda). Built using Java Spring Boot for the backend and a modern React + Vite for the frontend, it features live cloud synchronization via Supabase (PostgreSQL), ACID database transactions, secure authentication, and a robust cart management system.

SYSTEM ACCESS

App link: <https://final-exam-oop-2nd-yr.vercel.app/>

Files link: https://github.com/Do8Scared/final_exam_oop_2nd_yr

Download link: [finals_oop_download](#)

III. KEY FEATURES

- **Authentication:** Secure user login and registration system.
- **Transaction History:** View past orders dynamically pulled from the cloud database.
- **Detailed Receipts:** Terminal and UI receipts display

comprehensive customer and delivery details.

- **Live Menu System:** A fully synchronized menu displaying dynamic categories and options.

IV. PROJECT DOCUMENTATION

1. ENCAPSULATION

- Bundling data (variables) and methods into a single unit, and restricting direct access to sensitive data using private fields and public getters/setters.

```

1 package com.garahe.models;
2
3 /**
4  * Represents a single line item in a customer's shopping cart.
5  * Stores a MenuItem reference with a quantity and captures the unit price at the time of addition.
6  */
7 public class CartItem {
8
9     private MenuItem item;
10    private int quantity;
11    private double unitPrice;
12
13    /**
14     * Constructs a CartItem with the specified menu item and quantity.
15     * Records the unit price from the menu item at the time of creation.
16     *
17     * @param item the MenuItem being added to the cart
18     * @param quantity the number of units of this item
19     */
20    public CartItem(MenuItem item, int quantity) {
21        this.item = item;
22        this.quantity = quantity;
23        this.unitPrice = (item != null) ? item.getPrice() : 0;
24    }
25
26    /**
27     * Gets the menu item associated with this cart line.
28     *
29     * @return the MenuItem object
30     */
31    public MenuItem getItem() {
32        return item;
33    }
34
35    /**
36     * Sets a new menu item for this cart line and updates the unit price.
37     *
38     * @param item the new MenuItem
39     */
40    public void setItem(MenuItem item) {
41        this.item = item;
42        this.unitPrice = (item != null) ? item.getPrice() : 0;
43    }
44
45    /**
46     * Gets the quantity of items in this cart line.
47     *
48     * @return the quantity
49     */
50    public int getQuantity() {
51        return quantity;
52    }
53

```

```

53
54    /**
55     * Sets the quantity of items in this cart line.
56     *
57     * @param quantity the new quantity
58     */
59    public void setQuantity(int quantity) {
60        if (quantity <= 0) {
61            System.out.println("Error: Cart item quantity must be greater than zero.");
62            return;
63        }
64        this.quantity = quantity;
65    }
66
67    /**
68     * Gets the unit price of this item (captured at time of cart addition).
69     *
70     * @return the unit price in pesos
71     */
72    public double getUnitPrice() {
73        return unitPrice;
74    }
75

```


Figure 1.1 : CartItem.java

```

1 package com.garahe.models;
2
3 /**
4  * Represents a menu item in the ordering system.
5  * Contains pricing, inventory, and category information.
6  * Subclasses can override getSpecialDetails() for category-specific attributes.
7  */
8 public class MenuItem {
9
10     private final int id;
11     private String itemName;
12     private double price;
13     private int stockQuantity;
14     private final String category;
15
16     /**
17      * Constructs a MenuItem with a database-assigned ID.
18      * Used when loading items from the database.
19      *
20      * @param id the menu item ID assigned by the database
21      * @param itemName the name of the menu item
22      * @param price the price of the item
23      * @param stockQuantity the number of units in stock
24      * @param category the category this item belongs to
25      */
26     public MenuItem(int id, String itemName, double price, int stockQuantity, String category) {
27         this.id = id;
28         this.itemName = sanitizeText(itemName, "Unnamed Item");
29         this.price = Math.max(0, price);
30         this.stockQuantity = Math.max(0, stockQuantity);
31         this.category = sanitizeText(category, "Uncategorized");
32     }
33
34     /**
35      * Constructs a new MenuItem without a database ID.
36      * Used when creating brand new items before insertion into the database.
37      * The database will assign the real ID upon insertion.
38      *
39      * @param itemName the name of the menu item
40      * @param price the price of the item
41      * @param stockQuantity the number of units in stock
42      * @param category the category this item belongs to
43      */
44     public MenuItem(String itemName, double price, int stockQuantity, String category) {
45         this.id = 0;
46         this.itemName = sanitizeText(itemName, "Unnamed Item");
47         this.price = Math.max(0, price);
48         this.stockQuantity = Math.max(0, stockQuantity);
49         this.category = sanitizeText(category, "Uncategorized");
50     }
51

```

```

51
52     /**
53      * Gets the menu item ID.
54      *
55      * @return the item ID
56      */
57     public int getId() {
58         return id;
59     }
60
61     /**
62      * Gets the item name.
63      *
64      * @return the item name
65      */
66     public String getItemName() {
67         return itemName;
68     }
69
70     /**
71      * Gets the item price.
72      *
73      * @return the price in pesos
74      */
75     public double getPrice() {
76         return price;
77     }
78
79     /**
80      * Gets the current stock quantity.
81      *
82      * @return the number of units available
83      */
84     public int getStockQuantity() {
85         return stockQuantity;
86     }
87
88     /**
89      * Gets the category name.
90      *
91      * @return the category name
92      */
93     public String getCategory() {
94         return category;
95     }
96
97     /**
98      * Sets the item name with validation.
99      *
100     * @param itemName the new item name
101     */
102     public void setItemName(String itemName) {
103         this.itemName = sanitizeText(itemName, this.itemName);
104     }
105

```

```

106     /**
107      * Sets the item price. Negative values are rejected.
108      *
109      * @param newPrice the new price
110      */
111     public void setPrice(double newPrice) {
112         if (newPrice >= 0) {
113             this.price = newPrice;
114         } else {
115             System.out.println("Error: Price cannot be negative.");
116         }
117     }
118
119     /**
120      * Sets the stock quantity. Negative values are rejected.
121      *
122      * @param newQuantity the new stock quantity
123      */
124     public void setStockQuantity(int newQuantity) {
125         if (newQuantity >= 0) {
126             this.stockQuantity = newQuantity;
127         } else {
128             System.out.println("Error: Stock cannot be negative.");
129         }
130     }
131
132     /**
133      * Removes leading/trailing whitespace and null values in a string field.
134      *
135      * @param value the input string
136      * @param fallback the default value if input is null or empty
137      * @return the sanitized string
138      */
139     private String sanitizeText(String value, String fallback) {
140         if (value == null) {
141             return fallback;
142         }
143
144         String trimmed = value.trim();
145         return trimmed.isEmpty() ? fallback : trimmed;
146     }
147
148     /**
149      * Returns category-specific details for this menu item.
150      * Base implementation returns empty string. Subclasses override this method
151      * to provide special attributes (volume, spice level, protein, etc.).
152      *
153      * @return special details as a string, or empty string if none
154      */
155     public String getSpecialDetails() {
156         return "";
157     }
158 }
159

```

Figure 1.2 : MenuItem.java

The *CartItem* and *MenuItem* classes. A cart item's internal price or identity cannot be directly altered from the outside; the controlled `.setQuantity(int quantity)` method must be used. This method contains validation logic (if `(quantity <= 0)`) to prevent negative numbers.

```
1 package com.garahe.models;
2
3 /**
4  * Represents an Authenticated User in the system.
5  *
6  * OOP Pillars demonstrated:
7  * 1. Inheritance (Child): By using the `extends` keyword, this class inherits
8  *    the `id`, `name`, and `email` properties from the parent `Person` class,
9  *    allowing us to reuse code without duplicating it.
10 * 2. Encapsulation: The sensitive `password` field is kept strictly private
11 *    and accessed safely through getters/setters.
12 */
13 public class User extends Person {
14     private String password;
15
16     public User() {
17         super();
18     }
19
20     public User(int id, String name, String email, String password) {
21         super(id, name, email);
22         this.password = password;
23     }
24
25     public String getPassword() { return password; }
26     public void setPassword(String password) { this.password = password; }
27 }
28
```

Figure 1.3 : User.java

The image displays two side-by-side mobile application screens for user authentication, both featuring a dark theme and orange accents.

Left Screen: WELCOME BACK

- Header: GARAHE NI MATEICLA, WELCOME BACK, Close button (X).
- Buttons: LOG IN (orange), REGISTER (dark grey).
- EMAIL ADDRESS: Input field containing test@gmail.com.
- PASSWORD: Input field containing testing, with a toggle icon.
- Link: Forgot password? (orange).
- Button: LOG IN (orange).
- Footer: Don't have an account? Register (orange).

Right Screen: CREATE ACCOUNT

- Header: GARAHE NI MATEICLA, CREATE ACCOUNT, Close button (X).
- Buttons: LOG IN (dark grey), REGISTER (orange).
- FULL NAME: Input field containing test.
- EMAIL ADDRESS: Input field containing test@gmail.com.
- PASSWORD: Input field containing testing, with a toggle icon.
- CONFIRM PASSWORD: Input field containing testing.
- Button: CREATE ACCOUNT (orange).
- Footer: Already have an account? Log in (orange).

Figure 1.4 : User Registration/Login

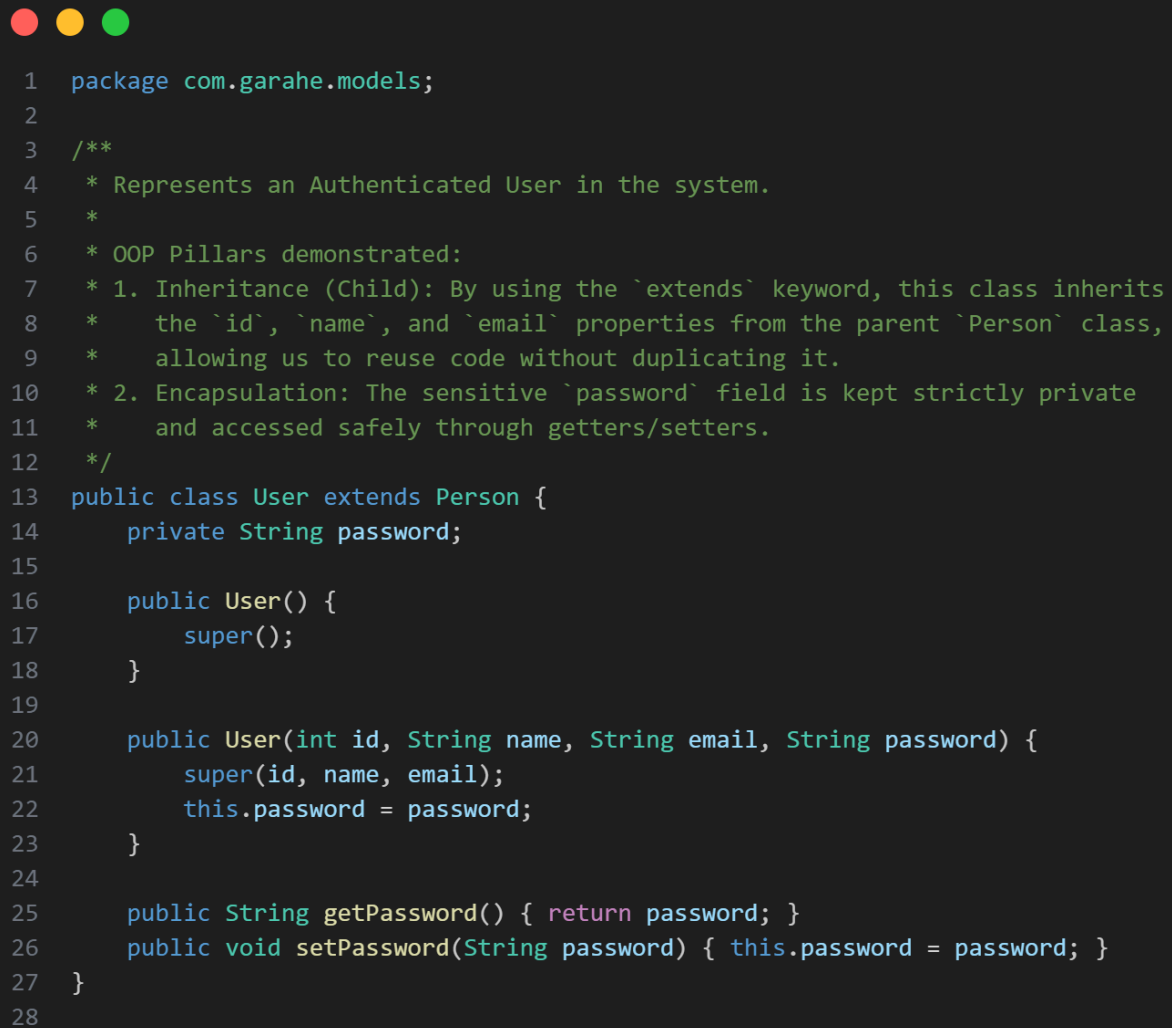
The User class (Authentication). Sensitive fields like password, email, and id are kept strictly private. They can only be accessed or modified through explicit getters and setters, protecting customer data integrity from external interference.

2. INHERITANCE

- Creating new child classes based on existing parent classes to promote code reusability using the `extends` keyword.

```
1 package com.garahe.models;
2
3 /**
4  * Represents a generic Person in the system.
5  *
6  * OOP Pillars demonstrated:
7  * 1. Abstraction: This class is `abstract`, meaning you cannot instantiate a raw Person.
8  *   It serves only as a foundational blueprint for more specific roles (like User).
9  * 2. Encapsulation: All fields are marked `private` to prevent unauthorized access,
10  *   and are only modifiable through public getters and setters.
11  * 3. Inheritance (Parent): This is the base class that other entities inherit from.
12  */
13 public abstract class Person {
14     private int id;
15     private String name;
16     private String email;
17
18     public Person() {}
19
20     public Person(int id, String name, String email) {
21         this.id = id;
22         this.name = name;
23         this.email = email;
24     }
25
26     public int getId() { return id; }
27     public void setId(int id) { this.id = id; }
28
29     public String getName() { return name; }
30     public void setName(String name) { this.name = name; }
31
32     public String getEmail() { return email; }
33     public void setEmail(String email) { this.email = email; }
34 }
35
```

Figure 1.5: Person.Java



```
1 package com.garahe.models;
2
3 /**
4  * Represents an Authenticated User in the system.
5  *
6  * OOP Pillars demonstrated:
7  * 1. Inheritance (Child): By using the `extends` keyword, this class inherits
8  *   the `id`, `name`, and `email` properties from the parent `Person` class,
9  *   allowing us to reuse code without duplicating it.
10 * 2. Encapsulation: The sensitive `password` field is kept strictly private
11 *   and accessed safely through getters/setters.
12 */
13 public class User extends Person {
14     private String password;
15
16     public User() {
17         super();
18     }
19
20     public User(int id, String name, String email, String password) {
21         super(id, name, email);
22         this.password = password;
23     }
24
25     public String getPassword() { return password; }
26     public void setPassword(String password) { this.password = password; }
27 }
28
```

Figure 1.6 : User.java

Authentication System: A base parent class called Person.java was created (containing universal traits like id, name, and email). The User.java class inherits from Person via the extends keyword and introduces the additional password trait specifically for system users. This prevents duplicating name and email code.


```

1 package com.garahe.models;
2
3 /**
4  * Represents a menu item in the ordering system.
5  * Contains pricing, inventory, and category information.
6  * Subclasses can override getSpecialDetails() for category-specific attributes.
7  */
8 public class MenuItem {
9
10     private final int id;
11     private String itemName;
12     private double price;
13     private int stockQuantity;
14     private final String category;
15
16     /**
17      * Constructs a MenuItem with a database-assigned ID.
18      * Used when loading items from the database.
19      *
20      * @param id the menu item ID assigned by the database
21      * @param itemName the name of the menu item
22      * @param price the price of the item
23      * @param stockQuantity the number of units in stock
24      * @param category the category this item belongs to
25      */
26     public MenuItem(int id, String itemName, double price, int stockQuantity, String category) {
27         this.id = id;
28         this.itemName = sanitizeText(itemName, "Unnamed Item");
29         this.price = Math.max(0, price);
30         this.stockQuantity = Math.max(0, stockQuantity);
31         this.category = sanitizeText(category, "Uncategorized");
32     }
33
34     /**
35      * Constructs a new MenuItem without a database ID.
36      * Used when creating brand new items before insertion into the database.
37      * The database will assign the real ID upon insertion.
38      *
39      * @param itemName the name of the menu item
40      * @param price the price of the item
41      * @param stockQuantity the number of units in stock
42      * @param category the category this item belongs to
43      */
44     public MenuItem(String itemName, double price, int stockQuantity, String category) {
45         this.id = 0;
46         this.itemName = sanitizeText(itemName, "Unnamed Item");
47         this.price = Math.max(0, price);
48         this.stockQuantity = Math.max(0, stockQuantity);
49         this.category = sanitizeText(category, "Uncategorized");
50     }
51

```

```

51
52     /**
53      * Gets the menu item ID.
54      *
55      * @return the item ID
56      */
57     public int getId() {
58         return id;
59     }
60
61     /**
62      * Gets the item name.
63      *
64      * @return the item name
65      */
66     public String getItemName() {
67         return itemName;
68     }
69
70     /**
71      * Gets the item price.
72      *
73      * @return the price in pesos
74      */
75     public double getPrice() {
76         return price;
77     }
78
79     /**
80      * Gets the current stock quantity.
81      *
82      * @return the number of units available
83      */
84     public int getStockQuantity() {
85         return stockQuantity;
86     }
87
88     /**
89      * Gets the category name.
90      *
91      * @return the category name
92      */
93     public String getCategory() {
94         return category;
95     }
96
97     /**
98      * Sets the item name with validation.
99      *
100     * @param itemName the new item name
101     */
102     public void setItemName(String itemName) {
103         this.itemName = sanitizeText(itemName, this.itemName);
104     }
105

```

```

106     /**
107      * Sets the item price. Negative values are rejected.
108      *
109      * @param newPrice the new price
110      */
111     public void setPrice(double newPrice) {
112         if (newPrice >= 0) {
113             this.price = newPrice;
114         } else {
115             System.out.println("Error: Price cannot be negative.");
116         }
117     }
118
119     /**
120      * Sets the stock quantity. Negative values are rejected.
121      *
122      * @param newQuantity the new stock quantity
123      */
124     public void setStockQuantity(int newQuantity) {
125         if (newQuantity >= 0) {
126             this.stockQuantity = newQuantity;
127         } else {
128             System.out.println("Error: Stock cannot be negative.");
129         }
130     }
131
132     /**
133      * Removes leading/trailing whitespace and null values in a string field.
134      *
135      * @param value the input string
136      * @param fallback the default value if input is null or empty
137      * @return the sanitized string
138      */
139     private String sanitizeText(String value, String fallback) {
140         if (value == null) {
141             return fallback;
142         }
143
144         String trimmed = value.trim();
145         return trimmed.isEmpty() ? fallback : trimmed;
146     }
147
148     /**
149      * Returns category-specific details for this menu item.
150      * Base implementation returns empty string. Subclasses override this method
151      * to provide special attributes (volume, spice level, protein, etc.).
152      *
153      * @return special details as a string, or empty string if none
154      */
155     public String getSpecialDetails() {
156         return "";
157     }
158 }

```

Figure 1.7: MenuItem.java

```

1  package com.garahe.models;
2
3  /**
4   * Represents an add-on menu item (condiment or solid food).
5   * Extends MenuItem with an additional isCondiment attribute.
6   */
7  public class AddOn extends MenuItem {
8
9      private boolean isCondiment;
10
11     /**
12      * Constructs an AddOn item with a type classification.
13      *
14      * @param id the menu item ID assigned by the database
15      * @param itemName the name of the add-on
16      * @param price the price of the add-on
17      * @param stockQuantity the number of units in stock
18      * @param category the category this item belongs to
19      * @param isCondiment true if this is a condiment/sauce, false if it's solid food
20      */
21     public AddOn(int id, String itemName, double price, int stockQuantity, String category, boolean isCondiment) {
22         super(id, itemName, price, stockQuantity, category);
23         this.isCondiment = isCondiment;
24     }
25
26     /**
27      * Checks if this add-on is a condiment or sauce.
28      *
29      * @return true if condiment, false if solid food
30      */
31     public boolean isCondiment() {
32         return isCondiment;
33     }
34
35     /**
36      * Sets the type classification for this add-on.
37      *
38      * @param condiment true for condiment/sauce, false for solid food
39      */
40     public void setCondiment(boolean condiment) {
41         this.isCondiment = condiment;
42     }
43
44     /**
45      * Returns the special details for this add-on (type information).
46      *
47      * @return a string representation of the add-on type
48      */
49     @Override
50     public String getSpecialDetails() {
51         return "[Type: " + (isCondiment() ? "Condiment/Sauce" : "Solid Food") + "]";
52     }
53 }
54

```

Figure 1.8 : AddOn.java (Child)

```

1 package com.garahe.models;
2
3 /**
4  * Represents an appetizer menu item with a piece count.
5  * Extends MenuItem with an additional piecesCount attribute.
6  */
7 public class Appetizer extends MenuItem {
8
9     private int piecesCount;
10
11     /**
12      * Constructs an Appetizer item with a specified piece count.
13      *
14      * @param id the menu item ID assigned by the database
15      * @param itemName the name of the appetizer
16      * @param price the price of the appetizer
17      * @param stockQuantity the number of units in stock
18      * @param category the category this item belongs to
19      * @param piecesCount the number of pieces in this appetizer serving
20      */
21     public Appetizer(int id, String itemName, double price, int stockQuantity, String category, int piecesCount) {
22         super(id, itemName, price, stockQuantity, category);
23         this.piecesCount = piecesCount;
24     }
25
26     /**
27      * Gets the number of pieces in this appetizer serving.
28      *
29      * @return the piece count
30      */
31     public int getPiecesCount() {
32         return piecesCount;
33     }
34
35     /**
36      * Sets the number of pieces. Must be greater than 0.
37      *
38      * @param piecesCount the new piece count
39      */
40     public void setPiecesCount(int piecesCount) {
41         if (piecesCount > 0) {
42             this.piecesCount = piecesCount;
43         } else {
44             System.out.println("Error: Piece count must be greater than 0.");
45         }
46     }
47
48     /**
49      * Returns the special details for this appetizer (piece count information).
50      *
51      * @return a string representation of the serving size
52      */
53     @Override
54     public String getSpecialDetails() {
55         return "[Serving: " + getPiecesCount() + " pieces]";
56     }
57 }
58

```

Figure 1.8.1: Appetizer (Child)

```

1  package com.garahe.models;
2
3  /**
4   * Represents a beverage menu item with volume specification.
5   * Extends MenuItem with an additional volumeInMl attribute.
6   */
7  public class Beverage extends MenuItem {
8
9      private int volumeInMl;
10
11      /**
12       * Constructs a Beverage item with a specified volume.
13       *
14       * @param id the menu item ID assigned by the database
15       * @param itemName the name of the beverage
16       * @param price the price of the beverage
17       * @param stockQuantity the number of units in stock
18       * @param category the category this item belongs to
19       * @param volumeInMl the volume of the beverage in milliliters
20       */
21      public Beverage(int id, String itemName, double price, int stockQuantity, String category, int volumeInMl) {
22          super(id, itemName, price, stockQuantity, category);
23          this.volumeInMl = volumeInMl;
24      }
25
26      /**
27       * Gets the volume of the beverage in milliliters.
28       *
29       * @return the volume in ml
30       */
31      public int getVolumeInMl() {
32          return volumeInMl;
33      }
34
35      /**
36       * Sets the volume of the beverage. Must be greater than 0 ml.
37       *
38       * @param volumeInMl the new volume in milliliters
39       */
40      public void setVolumeInMl(int volumeInMl) {
41          if (volumeInMl > 0) {
42              this.volumeInMl = volumeInMl;
43          } else {
44              System.out.println("Error: Beverage volume must be greater than 0 ml.");
45          }
46      }
47
48      /**
49       * Returns the special details for this beverage (volume information).
50       *
51       * @return a string representation of the volume
52       */
53      @Override
54      public String getSpecialDetails() {
55          return "[Volume: " + getVolumeInMl() + "ml]";
56      }
57  }
58

```

Figure 1.8.2 : Beverage(Child)

```

1  package com.garahe.models;
2
3  /**
4   * Represents a dessert menu item with a sweetness profile.
5   * Extends MenuItem with an additional sweetness attribute.
6   */
7  public class Dessert extends MenuItem {
8
9      private String sweetness;
10
11      /**
12       * Constructs a Dessert item with a specified sweetness profile.
13       *
14       * @param id the menu item ID assigned by the database
15       * @param itemName the name of the dessert
16       * @param price the price of the dessert
17       * @param stockQuantity the number of units in stock
18       * @param category the category this item belongs to
19       * @param sweetness the sweetness profile (e.g., "50% Sugar", "Low Sugar")
20       */
21      public Dessert(int id, String itemName, double price, int stockQuantity, String category, String sweetness) {
22          super(id, itemName, price, stockQuantity, category);
23          this.sweetness = sweetness;
24      }
25
26      /**
27       * Gets the sweetness profile of this dessert.
28       *
29       * @return the sweetness profile description
30       */
31      public String getSweetness() {
32          return sweetness;
33      }
34
35      /**
36       * Sets the sweetness profile of this dessert.
37       *
38       * @param sweetness the new sweetness profile
39       */
40      public void setSweetness(String sweetness) {
41          if (sweetness == null || sweetness.trim().isEmpty()) {
42              System.out.println("Error: Sweetness profile cannot be empty.");
43              return;
44          }
45          this.sweetness = sweetness.trim();
46      }
47
48      /**
49       * Returns the special details for this dessert (sweetness profile information).
50       *
51       * @return a string representation of the sweetness profile
52       */
53      @Override
54      public String getSpecialDetails() {
55          return "[Sweetness Profile: " + sweetness + "]";
56      }
57  }
58

```

Figure 1.8.3: Dessert(Child)

```

1  package com.garahe.models;
2
3  /**
4   * Represents a rice bowl menu item with a protein specification.
5   * Extends MenuItem with an additional mainProtein attribute.
6   */
7  public class RiceBowl extends MenuItem {
8
9      private String mainProtein;
10
11     /**
12      * Constructs a RiceBowl item with a specified protein.
13      *
14      * @param id the menu item ID assigned by the database
15      * @param itemName the name of the rice bowl
16      * @param price the price of the rice bowl
17      * @param stockQuantity the number of units in stock
18      * @param category the category this item belongs to
19      * @param mainProtein the main protein ingredient (e.g., Beef, Pork, Chicken)
20      */
21     public RiceBowl(int id, String itemName, double price, int stockQuantity, String category, String mainProtein) {
22         super(id, itemName, price, stockQuantity, category);
23         this.mainProtein = mainProtein;
24     }
25
26     /**
27      * Gets the main protein ingredient of this rice bowl.
28      *
29      * @return the protein name
30      */
31     public String getMainProtein() {
32         return mainProtein;
33     }
34
35     /**
36      * Sets the main protein ingredient of this rice bowl.
37      *
38      * @param mainProtein the new protein ingredient
39      */
40     public void setMainProtein(String mainProtein) {
41         this.mainProtein = mainProtein;
42     }
43
44     /**
45      * Returns the special details for this rice bowl (protein information).
46      *
47      * @return a string representation of the protein ingredient
48      */
49     @Override
50     public String getSpecialDetails() {
51         return "[Protein: " + getMainProtein() + "]";
52     }
53 }
54

```

Figure 1.8.4 : RiceBowl (Child)


```

1 package com.garahe.models;
2
3 /**
4  * Represents a soup menu item with spice level specification.
5  * Extends MenuItem with an additional isSpicy attribute.
6  */
7 public class Soup extends MenuItem {
8
9     private boolean isSpicy;
10
11     /**
12      * Constructs a Soup item with a specified spice level.
13      *
14      * @param id the menu item ID assigned by the database
15      * @param itemName the name of the soup
16      * @param price the price of the soup
17      * @param stockQuantity the number of units in stock
18      * @param category the category this item belongs to
19      * @param isSpicy true if the soup is prepared spicy, false otherwise
20      */
21     public Soup(int id, String itemName, double price, int stockQuantity, String category, boolean isSpicy) {
22         super(id, itemName, price, stockQuantity, category);
23         this.isSpicy = isSpicy;
24     }
25
26     /**
27      * Checks if this soup is prepared spicy.
28      *
29      * @return true if spicy, false otherwise
30      */
31     public boolean isSpicy() {
32         return isSpicy;
33     }
34
35     /**
36      * Sets the spice level for this soup.
37      *
38      * @param spicy true to prepare spicy, false for non-spicy
39      */
40     public void setSpicy(boolean spicy) {
41         this.isSpicy = spicy;
42     }
43
44     /**
45      * Returns the special details for this soup (preparation/spice level information).
46      *
47      * @return a string representation of the preparation style
48      */
49     @Override
50     public String getSpecialDetails() {
51         return "[Preparation: " + (isSpicy() ? "Spicy" : "Standard Non-Spicy") + "]";
52     }
53 }
54

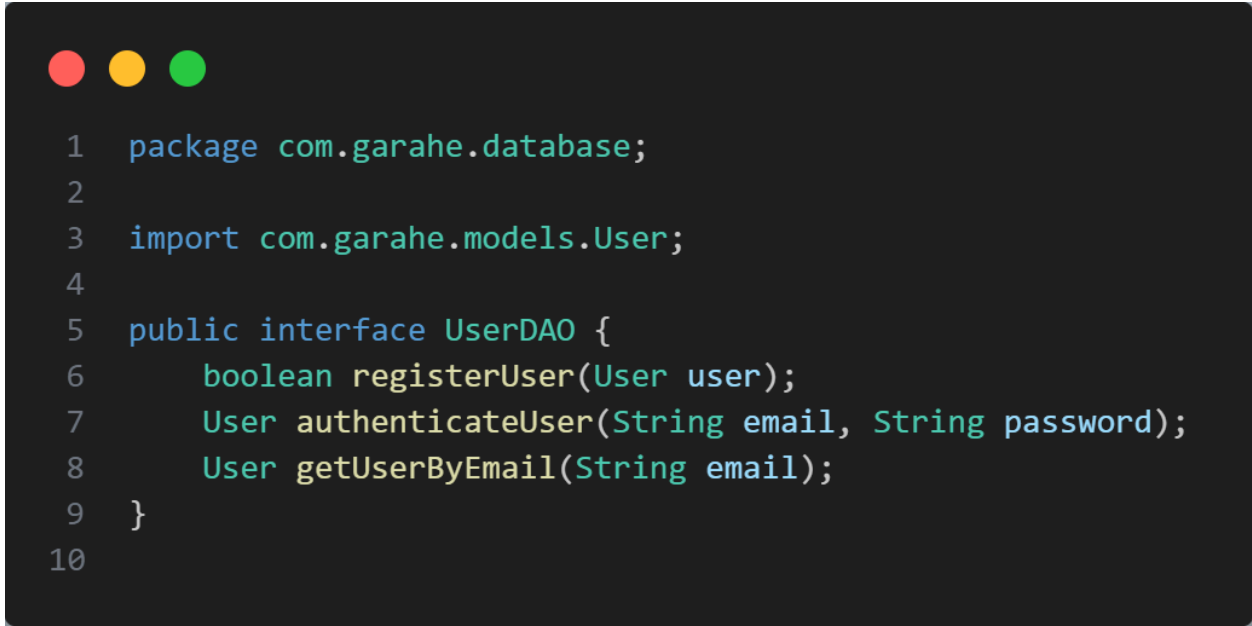
```

Figure 1.8.5 :Soup(Child)

Menu System: MenuItem.java is a Parent class. Specific Child classes like Beverage.java, Soup.java, and RiceBowl.java were created to extend MenuItem. This allows a Beverage to inherit all the pricing logic, but safely add its own unique property (e.g., volume in mL).

3. POLYMORPHISM

- The ability of different objects or methods to respond to the exact same call in their own unique way (Method Overriding and Interface implementation).



```
1 package com.garahe.database;
2
3 import com.garahe.models.User;
4
5 public interface UserDao {
6     boolean registerUser(User user);
7     User authenticateUser(String email, String password);
8     User getUserByEmail(String email);
9 }
10
```

Figure 1.9 :: UserDao

```

1 package com.garahe.database;
2
3 import com.garahe.models.User;
4 import java.sql.*;
5
6 /**
7  * Implementation of the UserDao interface.
8  *
9  * Part of the Data Access Object (DAO) pattern. This class is responsible for
10 * directly interacting with the Supabase PostgreSQL database. It abstracts away
11 * the complexity of JDBC connections, PreparedStatement mapping, and SQL execution.
12 */
13 public class UserDaoImpl implements UserDao {
14
15     /**
16      * Inserts a new user into the database.
17      *
18      * @param user The User object containing registration details.
19      * @return true if the database row was successfully inserted, false otherwise.
20      */
21     @Override
22     public boolean registerUser(User user) {
23         String sql = "INSERT INTO users (name, email, password) VALUES (?, ?, ?)";
24         try (Connection conn = DatabaseHelper.getConnection();
25             PreparedStatement pstmt = conn.prepareStatement(sql)) {
26
27             pstmt.setString(1, user.getName());
28             pstmt.setString(2, user.getEmail());
29             pstmt.setString(3, user.getPassword()); // For academic purposes, plaintext
30
31             return pstmt.executeUpdate() > 0;
32         } catch (SQLException e) {
33             System.out.println("Database Error during registration: " + e.getMessage());
34             return false;
35         }
36     }
37
38     /**
39      * Verifies the user credentials against the database.
40      *
41      * @param email The user's email address.
42      * @param password The user's plaintext password.
43      * @return A User object if the credentials match, or null if login fails.
44      */

```

```

45     @Override
46     public User authenticateUser(String email, String password) {
47         String sql = "SELECT id, name, email, password FROM users WHERE email = ?";
48         try (Connection conn = DatabaseHelper.getConnection();
49             PreparedStatement pstmt = conn.prepareStatement(sql)) {
50
51             pstmt.setString(1, email);
52             try (ResultSet rs = pstmt.executeQuery()) {
53                 if (rs.next()) {
54                     String storedPassword = rs.getString("password");
55                     if (storedPassword.equals(password)) {
56                         return new User(
57                             rs.getInt("id"),
58                             rs.getString("name"),
59                             rs.getString("email"),
60                             storedPassword
61                         );
62                     }
63                 }
64             }
65         } catch (SQLException e) {
66             System.out.println("Database Error during login: " + e.getMessage());
67         }
68         return null;
69     }
70
71     /**
72     * Retrieves a user from the database strictly by their email.
73     * Used for mapping relationships (like fetching user_id for a transaction).
74     *
75     * @param email The user's email address.
76     * @return The User object if found, null otherwise.
77     */
78     @Override
79     public User getUserByEmail(String email) {
80         String sql = "SELECT id, name, email, password FROM users WHERE email = ?";
81         try (Connection conn = DatabaseHelper.getConnection();
82             PreparedStatement pstmt = conn.prepareStatement(sql)) {
83
84             pstmt.setString(1, email);
85             try (ResultSet rs = pstmt.executeQuery()) {
86                 if (rs.next()) {
87                     return new User(
88                         rs.getInt("id"),
89                         rs.getString("name"),
90                         rs.getString("email"),
91                         rs.getString("password")
92                     );
93                 }
94             }
95         } catch (SQLException e) {
96             System.out.println("Database Error fetching user by email: " + e.getMessage());
97         }
98         return null;
99     }
100 }

```

Figure 1. : UserDAOImpl

Database Implementation: The UserDAO interface acts as a polymorphic contract. UserDAOImpl implements this interface dynamically. The AuthController only talks to the UserDAO interface, allowing Java to dynamically execute the implementation.

```
1 package com.garahe.orders;
2
3 /**
4  * Interface for order fulfillment types (e.g., Delivery, Pick-Up).
5  * Defines the contract for order-type-specific fees and behaviors.
6  *
7  * Renamed from OrderDAO to OrderType to avoid naming collision with
8  * database.TransactionDAO and to accurately reflect what this interface represents:
9  * an order TYPE, not a data access object.
10 */
11 public interface OrderType {
12
13     /**
14      * Returns the additional fee associated with this order type.
15      *
16      * @return the additional fee amount
17      */
18     double getAdditionalFee();
19
20 }
21
```

Figure 1.: OrderType.java



```
1 package com.garahe.orders;
2
3 /**
4  * Represents a delivery order with a fixed delivery fee.
5  * Implements the OrderType interface for order type-specific behavior.
6  */
7 public class DeliveryOrder implements OrderType {
8
9     /**
10      * Initializes a delivery order.
11      */
12     public DeliveryOrder() {
13     }
14
15     /**
16      * Returns the delivery fee for this order.
17      *
18      * @return 50.00
19      */
20     @Override
21     public double getAdditionalFee() {
22         return 50.00;
23     }
24 }
25
```

Figure 1.: DeliveryOrder

```

1  package com.garahe.orders;
2
3  /**
4   * Represents a pick-up order with no additional fees.
5   * Implements the OrderType interface for order type-specific behavior.
6   */
7  public class PickupOrder implements OrderType {
8
9      /**
10       * Initializes a pick-up order.
11       * Pick-up orders have no additional fee since service is on premises.
12       */
13     public PickupOrder() {
14     }
15
16     /**
17      * Returns the additional fee for a pick-up order.
18      *
19      * @return 0.00 (no fee for pick-up service)
20      */
21     @Override
22     public double getAdditionalFee() {
23         return 0.00;
24     }
25 }
26

```

Figure 1.: PickupOrder

The Order Interface: The OrderType interface. Both DeliveryOrder and PickupOrder implement this interface. When the checkout engine asks for .getAdditionalFee(), Delivery dynamically calculates and returns 50.00, while Pick-up returns 0.00.

```

94 > public String getSpecialDetails() { return ""; }
97 }
98

```

Figure 1.: *getSpecialDetails()*

```

44 /**
45  * Returns the special details for this soup (preparation/spice level information).
46  *
47  * @return a string representation of the preparation style
48  */
49 @Override
50 public String getSpecialDetails() {
51     return "[Preparation: " + (isSpicy() ? "Spicy" : "Standard Non-Spicy") + "];"
52 }
53 }
54

```

Figure 1.: *Soup.java*

Dynamic Menu Data: The `getSpecialDetails()` method. The parent `MenuItem` has an empty method, but every child class `@Override`s it. Soups return `[Preparation: Non-Spicy]`, drinks return `[Volume: 500ml]`.

4. ABSTRACTION

- Hiding complex background implementation details and showing only the essential features to the user (or other parts of the code).



```
1  String txnId = TransactionDAO.processCheckout(  
2      cart,  
3      payload.getOrderType(),  
4      payload.getPaymentMethod(),  
5      payload.getAdditionalFee(),  
6      payload.getEmail(),  
7      payload.getCustomerName(),  
8      payload.getContactNumber(),  
9      payload.getDeliverTo(),  
10     payload.getNotes()  
11 );  
12
```

Figure 1:

```
/**  
 * Processes a customer checkout transaction with ACID guarantees.  
 * Inserts transaction records, allocates items to the order, updates stock  
 * levels,  
 * and logs the transaction to the audit trail. Rolls back all changes if any  
 * step fails.  
 *  
 * @param cart        the list of CartItems being purchased  
 * @param orderType    the order type (Dine-In or Take-Out)  
 * @param paymentMethod the payment method (Cash, GCash, Maya)  
 * @param amountTendered the cash amount provided by the customer  
(for Cash  
 *      payments)  
 * @param additionalFee the additional fee (e.g., delivery fee)  
 * @param email         the email of the logged-in user  
 * @param customerName  the name of the customer  
 * @param contactNumber the contact number  
 * @param deliverTo     the delivery address  
 * @param notes         any additional notes  
 * @return the transaction ID if successful, null otherwise
```

```

    */
    public static String processCheckout(List<CartItem> cart, String orderType,
String paymentMethod,
        double additionalFee, String email, String customerName, String
contactNumber, String deliverTo, String notes) {
        if (cart == null || cart.isEmpty()) {
            System.out.println("Transaction Failed: Cart is empty.");
            return null;
        }

        Integer userId = null;
        if (email != null && !email.isEmpty()) {
            User u = userDao.getUserByEmail(email);
            if (u != null) {
                userId = u.getId();
            }
        }

        String txnId = "TXN-" + UUID.randomUUID().toString().substring(0,
8).toUpperCase() + "-"
            + orderType.toUpperCase();
        double subtotal = 0;
        for (CartItem item : cart) {
            subtotal += item.getSubtotal();
        }

        subtotal = Math.round(subtotal * 100.0) / 100.0;
        double grandTotal = Math.round((subtotal + additionalFee) * 100.0) /
100.0;
        double amountTendered = grandTotal;
        double changeDue = 0.00;

        String insertTxnSql = "INSERT INTO transactions (transaction_id,
order_type, payment_method, total_amount, amount_tendered,
change_due, user_id, customer_name, contact_number, deliver_to, notes)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
        String insertItemsSql = "INSERT INTO transaction_items (transaction_id,
menu_item_id, quantity, subtotal) VALUES (?, ?, ?, ?)";
        String updateStockSql = "UPDATE menu_items SET stock_quantity =
stock_quantity - ? WHERE id = ?";
        String lockStockSql = "SELECT stock_quantity FROM menu_items WHERE id
= ? AND is_active = TRUE FOR UPDATE";

        Connection conn = null;
        try {

```

```

conn = DatabaseHelper.getConnection();
conn.setAutoCommit(false);

try (PreparedStatement pstmtTxn =
conn.prepareStatement(insertTxnSql)) {
    pstmtTxn.setString(1, txnId);
    pstmtTxn.setString(2, orderType);
    pstmtTxn.setString(3, paymentMethod);
    pstmtTxn.setDouble(4, grandTotal);
    pstmtTxn.setDouble(5, amountTendered);
    pstmtTxn.setDouble(6, changeDue);
    if (userId != null) pstmtTxn.setInt(7, userId); else pstmtTxn.setNull(7,
Types.INTEGER);
    pstmtTxn.setString(8, customerName);
    pstmtTxn.setString(9, contactNumber);
    pstmtTxn.setString(10, deliverTo);
    pstmtTxn.setString(11, notes);
    pstmtTxn.executeUpdate();
}

try (PreparedStatement pstmtItems =
conn.prepareStatement(insertItemsSql);
    PreparedStatement pstmtStock =
conn.prepareStatement(updateStockSql);
    PreparedStatement pstmtLock =
conn.prepareStatement(lockStockSql)) {

    for (CartItem c : cart) {
        if (c == null || c.getItem() == null) {
            throw new SQLException("Cart contains an invalid line item.");
        }
        if (c.getQuantity() <= 0) {
            throw new SQLException(
                "Cart contains an invalid quantity for item ID " +
c.getItem().getId() + ".");
        }

        pstmtLock.setInt(1, c.getItem().getId());
        try (ResultSet stockRs = pstmtLock.executeQuery()) {
            if (!stockRs.next()) {
                throw new SQLException("Item ID " + c.getItem().getId() + " is no
longer available.");
            }

            int currentStock = stockRs.getInt("stock_quantity");

```

```

        if (currentStock < c.getQuantity()) {
            throw new SQLException("Insufficient stock for " +
c.getItem().getItemName()
            + ". Requested " + c.getQuantity() + ", available " +
currentStock + ".");
        }
    }
}

```

```

pstmtItems.setString(1, txnId);
pstmtItems.setInt(2, c.getItem().getId());
pstmtItems.setInt(3, c.getQuantity());
pstmtItems.setDouble(4, c.getSubtotal());
pstmtItems.addBatch();

```

```

pstmtStock.setInt(1, c.getQuantity());
pstmtStock.setInt(2, c.getItem().getId());
pstmtStock.addBatch();
}

```

```

pstmtItems.executeBatch();
pstmtStock.executeBatch();
}

```

```

try {
    StringBuilder itemsJson = new StringBuilder();
    itemsJson.append("[");
    boolean first = true;
    for (CartItem c : cart) {
        if (!first)
            itemsJson.append(',');
        Map<String, Object> itemMap = new LinkedHashMap<>();
        itemMap.put("id", c.getItem().getId());
        itemMap.put("name", c.getItem().getItemName());
        itemMap.put("qty", c.getQuantity());
        itemsJson.append(JsonUtil.buildJsonObject(itemMap));
        first = false;
    }
    itemsJson.append("]");
}

```

```

Map<String, Object> detailsMap = new LinkedHashMap<>();
detailsMap.put("txnId", txnId);
detailsMap.put("orderType", orderType);
detailsMap.put("total", grandTotal);

```

```

String detailsBase = JsonUtil.buildJsonObject(detailsMap);

```

```

        // Insert items array into the details JSON
        String details = detailsBase.substring(0, detailsBase.length() - 1) +
        ", \"items\" : " + itemsJson + "}";

        DatabaseHelper.insertAudit(conn, "terminal", "ORDER_CHECKOUT",
        txnId, details);
    } catch (SQLException ex) {
        System.out.println("Failed to write audit log: " + ex.getMessage());
        conn.rollback();
        return null;
    }

    conn.commit();

    printUnifiedReceipt(txnId, cart, subtotal, additionalFee, grandTotal,
    paymentMethod, amountTendered,
    changeDue, customerName, contactNumber, deliverTo, notes);
    return txnId;

} catch (SQLException e) {
    if (conn != null) {
        try {
            conn.rollback();
        } catch (SQLException ex) {
        }
    }
    if (e.getMessage().contains("check_stock_positive")) {
        System.out.println(
            ">> [TRANSACTION FAILED] Someone just bought the last item!
Insufficient stock in the cloud.");
    } else {
        System.out.println(">> [TRANSACTION FAILED] Database Error: " +
        e.getMessage());
    }
    return null;
} finally {
    if (conn != null) {
        try {
            conn.setAutoCommit(true);
            conn.close();
        } catch (SQLException e) {
            System.out.println("Error closing transaction connection: " +
            e.getMessage());
        }
    }
}
}

```

```

    }
}

```

Figure 1.: *ProcessCheckout*

Order DAO: Our Spring Boot API Controllers (*OrderController.java* and *AuthController.java*) have no idea that PostgreSQL exists. When a customer adds an item to their cart and completes checkout, the UI simply calls a clean, readable method like *TransactionDAO.processCheckout()*. The DAO completely hides the complex mechanics of *PreparedStatement* and *ResultSet* objects, connection handling, ACID transactions, and network errors away from the front-end.

```

1  /**
2   * Authenticates an existing user for login.
3   *
4   * @param payload The login credentials containing email and password.
5   * @return ResponseEntity containing the User object (with password stripped for security) or an unauthorized error.
6   */
7  @PostMapping("/login")
8  public ResponseEntity<> login(@RequestBody LoginPayload payload) {
9      User user = userDao.authenticateUser(payload.getEmail(), payload.getPassword());
10     if (user != null) {
11         // Remove sensitive info before returning to client
12         user.setPassword(null);
13         return ResponseEntity.ok(user);
14     } else {
15         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("{\"error\": \"Invalid email or password\"}");
16     }
17 }
18

```

Figure 1.:

Figure 1.:

Auth DAO: The *AuthController* securely registers and logs in users by calling *userDAO.authenticateUser()*. It doesn't write SQL queries; it relies entirely on the Abstraction provided by the *UserDAO* interface.

5. EXCEPTION HANDLING

- Capturing and handling runtime errors (like database connection drops) using try-catch blocks to prevent the application from crashing unexpectedly.

```

MenuManager.java

public static void addMenuItem(MenuItem item, String specialAttribute, String actor) { 1 usage
    String sql = "INSERT INTO menu_items (item_name, price, stock_quantity, category, special_attribute) VALUES (?, ?, ?, ?, ?) RETURNING id";

    Connection conn = null;
    try {
        conn = DatabaseHelper.getConnection();
        conn.setAutoCommit(false);

        Integer createdId = null;
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, item.getItemName());
            pstmt.setDouble(2, item.getPrice());
            pstmt.setInt(3, item.getStockQuantity());
            pstmt.setString(4, item.getCategory());
            pstmt.setString(5, specialAttribute);

            try (java.sql.ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    createdId = rs.getInt("id");
                }
            }
        }

        if (createdId != null) {
            Map<String, Object> details = new LinkedHashMap<>();
            details.put("id", createdId);
            details.put("name", item.getItemName());
            details.put("price", item.getPrice());
            details.put("stock", item.getStockQuantity());
            details.put("category", item.getCategory());
            details.put("special_attribute", specialAttribute);

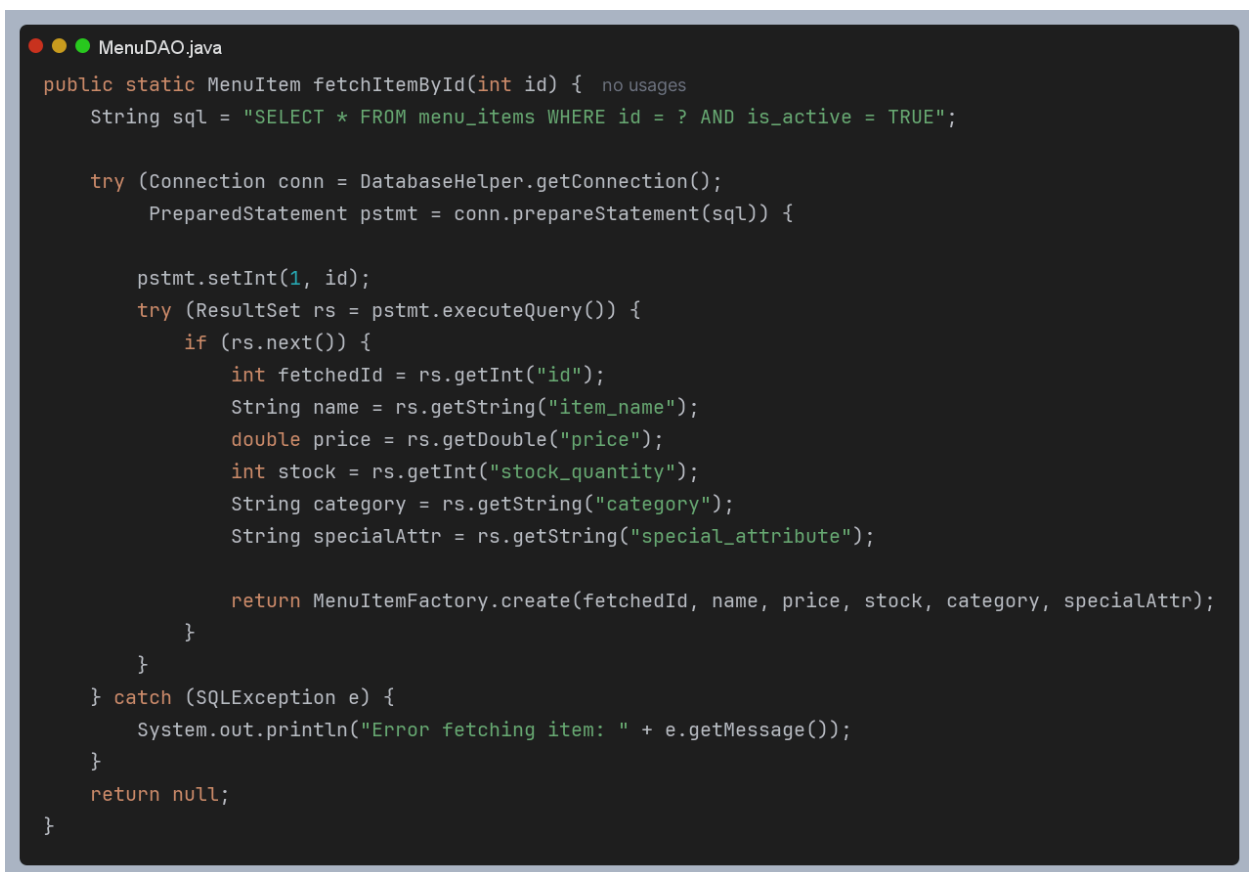
            DatabaseHelper.insertAudit(conn, actor, "MENU_ITEM_CREATED",
                String.valueOf(createdId), JsonUtil.buildJsonObject(details));

            conn.commit();
            System.out.println("\n[SYSTEM] Success! " + item.getItemName() + " was added to the live database.");
        } else {
            conn.rollback();
            System.out.println("\n[SYSTEM ERROR] Menu item insert did not return a new ID.");
        }
    } catch (SQLException e) {
        // Fix: always rollback on exception (was missing before)
        if (conn != null) {
            try { conn.rollback(); } catch (SQLException ex) { /* rollback best-effort */ }
        }
        System.out.println("\n[SYSTEM ERROR] Could not add the menu item to the database.");
        System.out.println("Error details: " + e.getMessage());
    } finally {
        if (conn != null) {
            try {
                conn.setAutoCommit(true);
                conn.close();
            } catch (SQLException e) {
                System.out.println("Error closing connection: " + e.getMessage());
            }
        }
    }
}

```

Figure 1: try-catch \ .rollback

Database Transactions: In our DAOs (e.g., TransactionDAO), all PostgreSQL queries are wrapped in try-catch blocks. If the database connection drops or a query fails mid-checkout, the SQLException is caught, a .rollback() is safely triggered to ensure no money or stock is lost, and an error message is printed gracefully instead of the system crashing.



```
MenuDAO.java
public static MenuItem fetchItemById(int id) { no usages
    String sql = "SELECT * FROM menu_items WHERE id = ? AND is_active = TRUE";

    try (Connection conn = DatabaseHelper.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, id);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                int fetchedId = rs.getInt("id");
                String name = rs.getString("item_name");
                double price = rs.getDouble("price");
                int stock = rs.getInt("stock_quantity");
                String category = rs.getString("category");
                String specialAttr = rs.getString("special_attribute");

                return MenuItemFactory.create(fetchedId, name, price, stock, category, specialAttr);
            }
        }
    } catch (SQLException e) {
        System.out.println("Error fetching item: " + e.getMessage());
    }
    return null;
}
```

Figure 1.: try-catch] DatabaseHelper.getConnection()

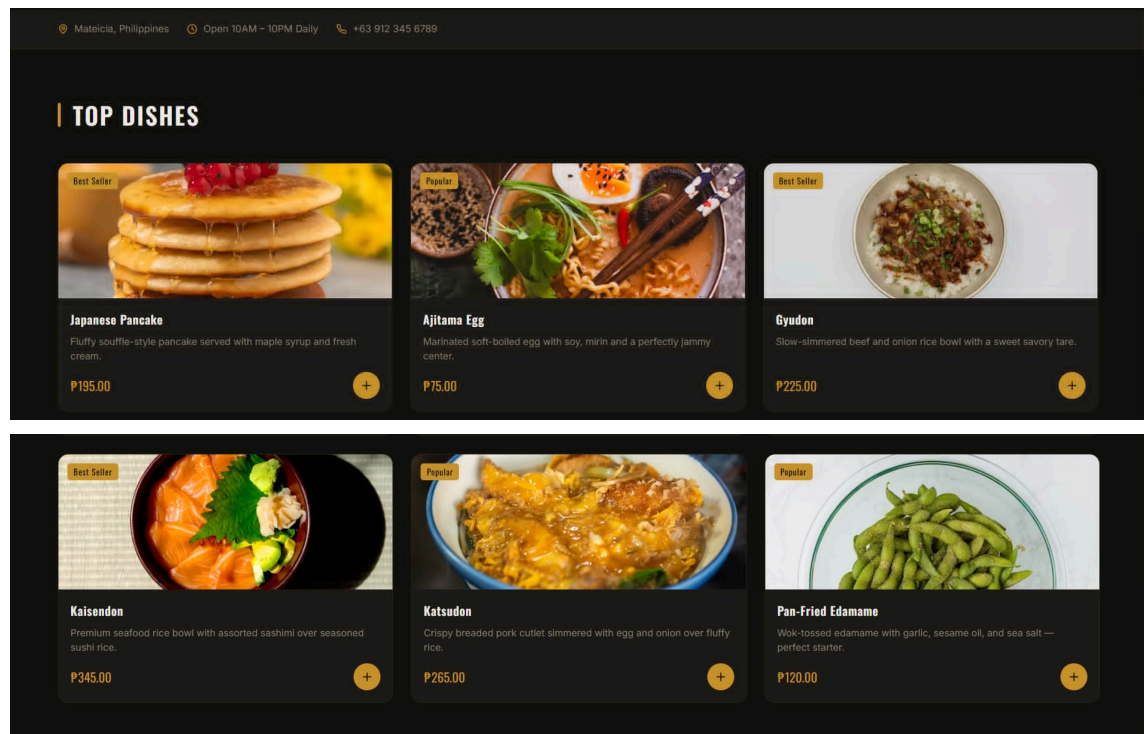
Try-With-Resources: We utilize modern Java try-with-resources blocks (e.g., try (Connection conn = DatabaseHelper.getConnection())) which guarantees that sensitive database connections are automatically

closed and memory is freed, even if an error occurs.

V. ADDITIONAL INFORMATION

1. WEBSITE SCREENSHOTS

Figure 2.0 : Homepage of Garahe ni Mateicla Website



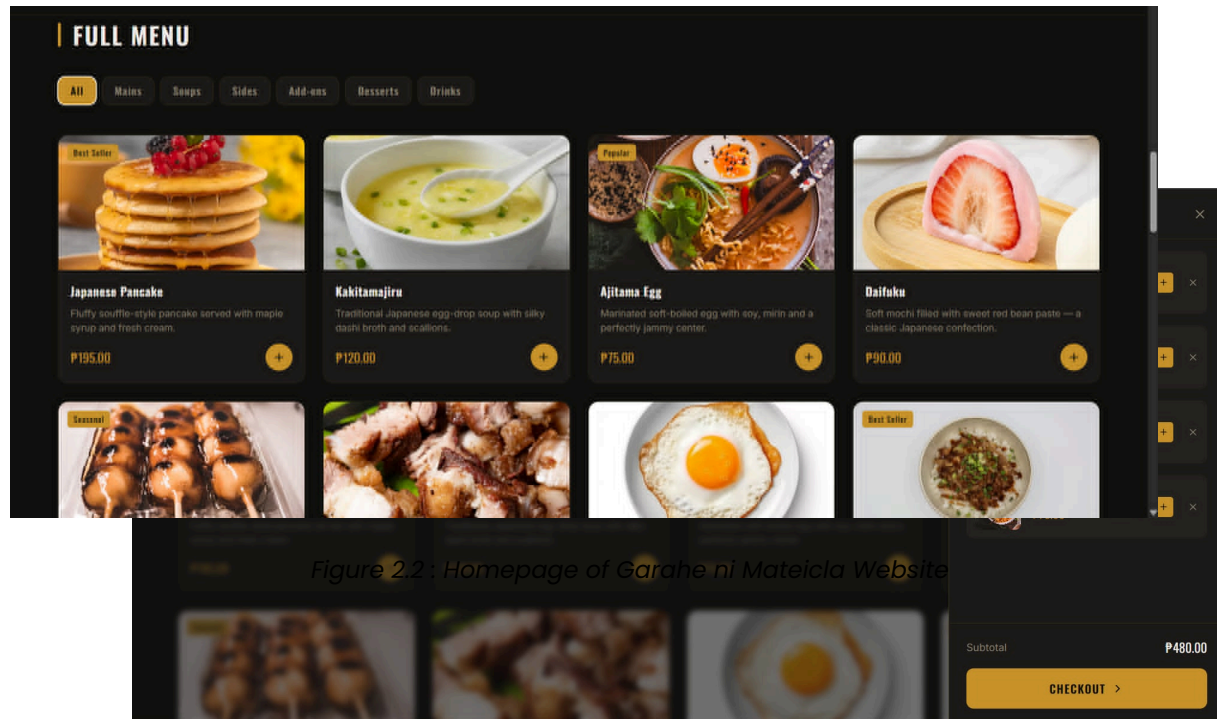


Figure 2.2 : Homepage of Garahe ni Mateicla Website

Figure 2.1 : Homepage of Garahe ni Mateicla Website

DELIVERY DETAILS

1 Details

2 Review

FULL NAME

Vincent

EMAIL

mirexellevincentc.piol@gmail.com

PHONE NUMBER

1234567

DELIVERY ADDRESS

pup

ORDER NOTES (OPTIONAL)

none

PAYMENT METHOD


Cash on Delivery


GCash


Card

Figure 3 : Delivery Details panel of Garahe ni Mateicla Website

<

ORDER SUMMARY

×

1

Details

2

Review

DELIVERY TO

Vincent

pup

1234567



Japanese Pancake

x1

P195.00



Kakitamajiru

x1

P120.00



Daifuku

x1

P90.00



Ajitama Egg

x1

P75.00

Subtotal

P480.00

Delivery Fee

P50.00

TOTAL

P530.00

PLACE ORDER >

Figure 4 : Order Summary panel of Garahe ni Mateicla Website

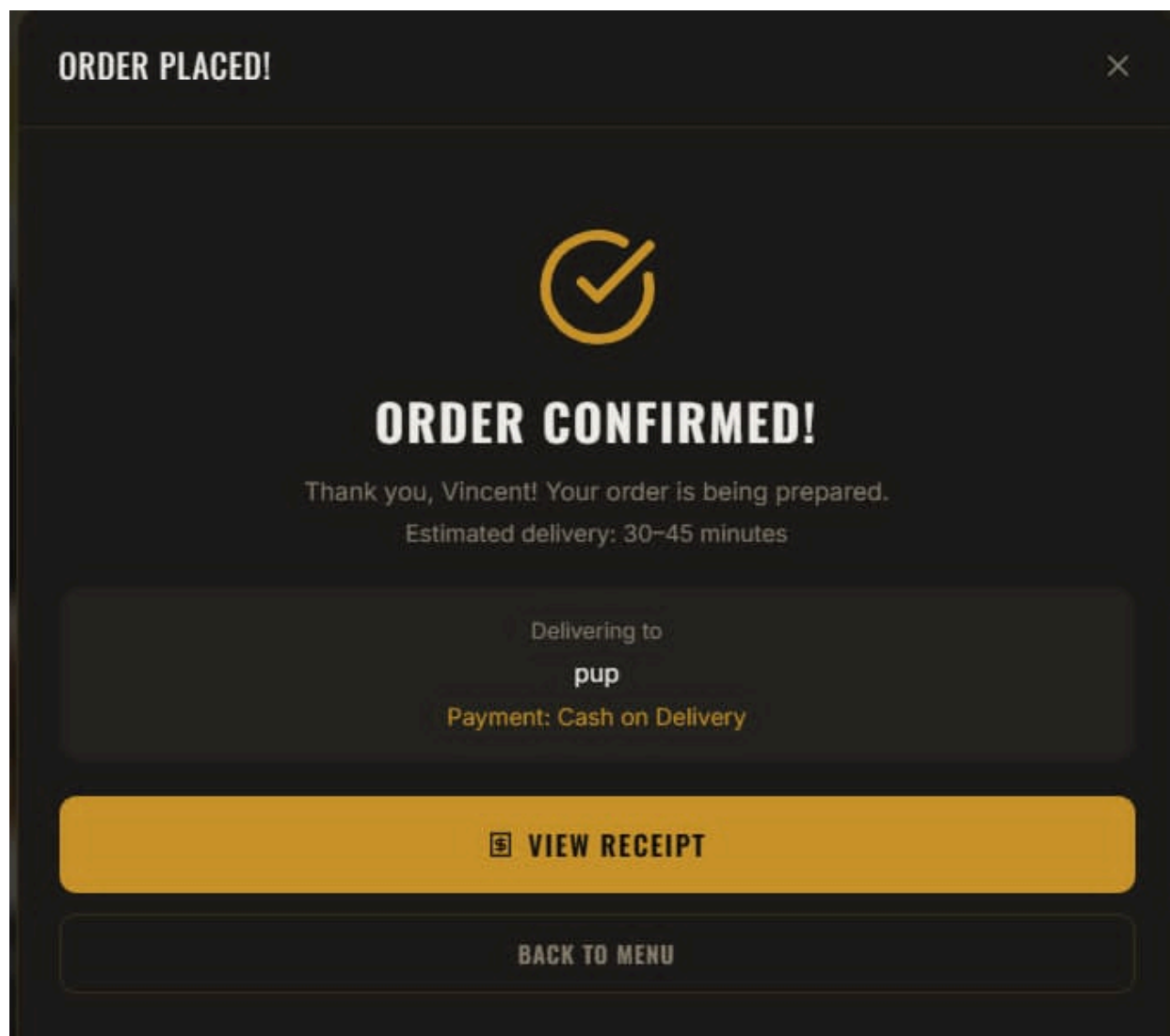


Figure 4.1: Order confirmation panel of Garahe ni Mateicla Website

✓ ORDER RECEIPT

PRINT

X

OFFICIAL RECEIPT

GARAHE NI MATEICLA

Mateicla, Philippines - +63 912 345 6789

Open 10AM - 10PM Daily

Order No.

#TXN-72C3FDB6-DELIVERY

Date

June 14, 2026

Time

09:49 PM

Customer

Vincent

Contact

1234567

Deliver To

pup

Notes

none

ITEM	QTY	AMOUNT
Japanese Pancake	x1	₱195.00
Kakitamajiru	x1	₱120.00
Daifuku	x1	₱90.00
Ajitama Egg	x1	₱75.00

Subtotal

₱480.00

Delivery Fee

₱50.00

TOTAL

₱530.00

Payment

Cash on Delivery

SALAMATI THANK YOU!

Please keep this receipt as proof of your order.

For concerns call +63 912 345 6789.

Figure 4.2 : Order Receipt of Garahe ni Mateicla Website

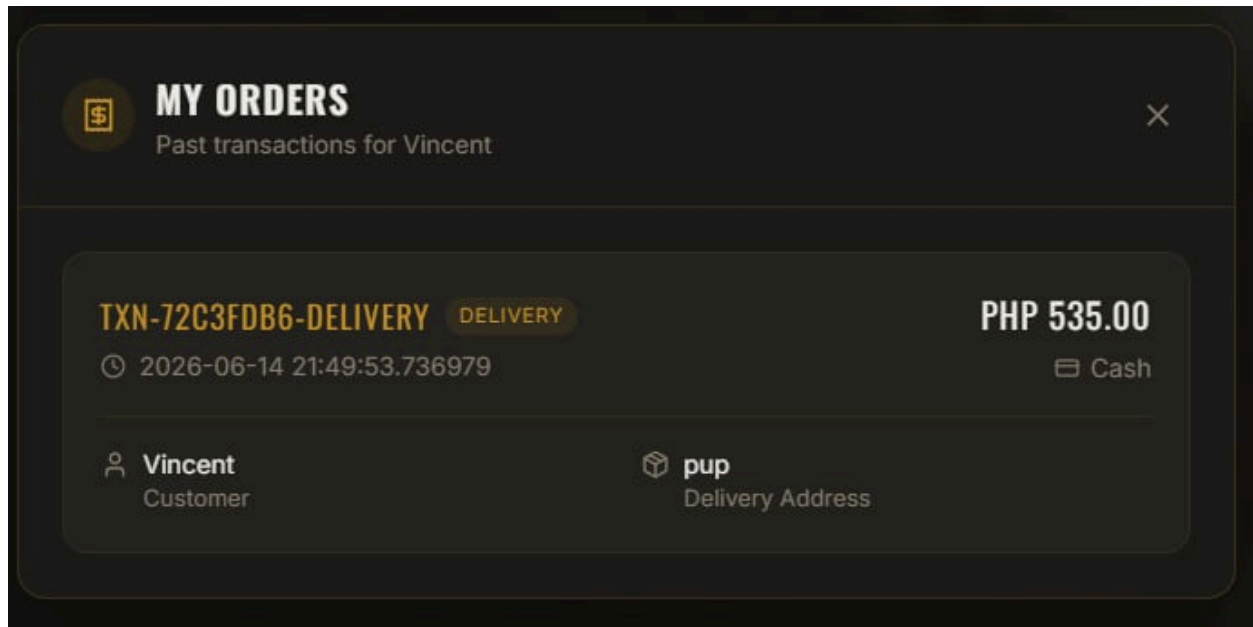


Figure 4.3 : Order Tracker of Garahe ni Mateicla Website

2. SUPASBASE TABLE SCREENSHOTS

id	int4	item_name	varchar	price	numeric	stock_quantity	int4	category	varchar	special_attribute	varchar	is_active	bool	created_at	timestampz
1		Kyuri Taukemono		120.00		26		Appetizer		1		TRUE		2026-06-14 12:02:18.205261+00	
2		Pan-Fried Edamame		130.00		92		Appetizer		1		TRUE		2026-06-14 12:02:18.205261+00	
3		Chicken Yakitori		190.00		38		Appetizer		3		TRUE		2026-06-14 12:02:18.205261+00	
4		Ajitama Egg		45.00		47		Appetizer		1		TRUE		2026-06-14 12:02:18.205261+00	
5		Pork Chashu		180.00		39		Appetizer		3		TRUE		2026-06-14 12:02:18.205261+00	
6		Miso Soup		60.00		79		Soup		Non-Spicy		TRUE		2026-06-14 12:02:18.205261+00	
7		Kabocha Soup		95.00		30		Soup		Non-Spicy		TRUE		2026-06-14 12:02:18.205261+00	
8		Kakitamajiru		75.00		40		Soup		Non-Spicy		TRUE		2026-06-14 12:02:18.205261+00	
9		Japanese Corn Soup		85.00		35		Soup		Non-Spicy		TRUE		2026-06-14 12:02:18.205261+00	
10		Unagi Don		495.00		6		Rice Bowl		Eel		TRUE		2026-06-14 12:02:18.205261+00	
11		Gyudon		265.00		22		Rice Bowl		Beef		TRUE		2026-06-14 12:02:18.205261+00	
12		Oyakodon		240.00		40		Rice Bowl		Chicken		TRUE		2026-06-14 12:02:18.205261+00	
13		Katsudon		250.00		34		Rice Bowl		Pork		TRUE		2026-06-14 12:02:18.205261+00	
14		Tendon		320.00		25		Rice Bowl		Shrimp		TRUE		2026-06-14 12:02:18.205261+00	
15		Kaisendon		450.00		8		Rice Bowl		Seafood		TRUE		2026-06-14 12:02:18.205261+00	
16		Mochi		90.00		59		Dessert		Standard		TRUE		2026-06-14 12:02:18.205261+00	
17		Daifuku		110.00		40		Dessert		High Sugar		TRUE		2026-06-14 12:02:18.205261+00	

Figure 5.0 : MenuItem Table

id	int4	name	varchar	email	varchar	password	varchar	created_at	timestampz
1		Vincent		mirexellevincentc.piol@gmail.com		mirexellevincent		2026-06-14 12:02:18.205261+00	
2		Ivan		kirotakaYD@gmail.com		kirotaka		2026-06-14 12:19:22.803118+00	

Figure 5.1: User Table

on all displayed rows Ctrl A

☐	☒ id int4	transaction_id varchar	order_type varchar	payment_method varchar	total_amount numeric	amount_tendered numeric
☐	1	TXN-78D01100-DINE-IN	Dine-In	Cash	120.00	200.00
☐	2	TXN-2C0C66E7-DINE-IN	Dine-In	Cash	120.00	200.00
☐	3	TXN-4BE54943-DINE-IN	Dine-In	Maya	390.00	NULL
☐	4	TXN-569AEB9D-DINE-IN	Dine-In	Cash	120.00	3000.00
☐	5	TXN-0B438355-PICK-UP	Pick-Up	GCash	120.00	120.00
☐	6	TXN-58411014-PICK-UP	Pick-Up	Maya	360.00	360.00
☐	7	TXN-5412CB2C-PICK-UP	Pick-Up	Maya	1130.00	1130.00
☐	8	TXN-D3CC6288-PICK-UP	Pick-Up	GCash	250.00	250.00
☐	9	TXN-6F28CBE5-PICK-UP	Pick-Up	GCash	120.00	120.00
☐	10	TXN-E7DF3DD3-PICK-UP	Pick-Up	GCash	1.00	1.00
☐	11	TXN-A946ABF0-DELIVERY	Delivery	Cash	170.00	170.00
☐	12	TXN-91A06860-DELIVERY	Delivery	Card	180.00	180.00
☐	13	TXN-9D3C86A5-DELIVERY	Delivery	Card	230.00	230.00
☐	14	TXN-E904F2DA-DELIVERY	Delivery	GCash	180.00	180.00
☐	15	TXN-A5529239-DELIVERY	Delivery	Cash	5035.00	5035.00
☐	16	TXN-BCB8B074-DELIVERY	Delivery	Cash	360.00	360.00
☐	17	TXN-72C3FDB6-DELIVERY	Delivery	Cash	535.00	535.00

customer_name varchar	contact_number varchar	deliver_to varchar	notes text
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
Vincent	1234567	pup	none

Figure 5.2: Transaction Table

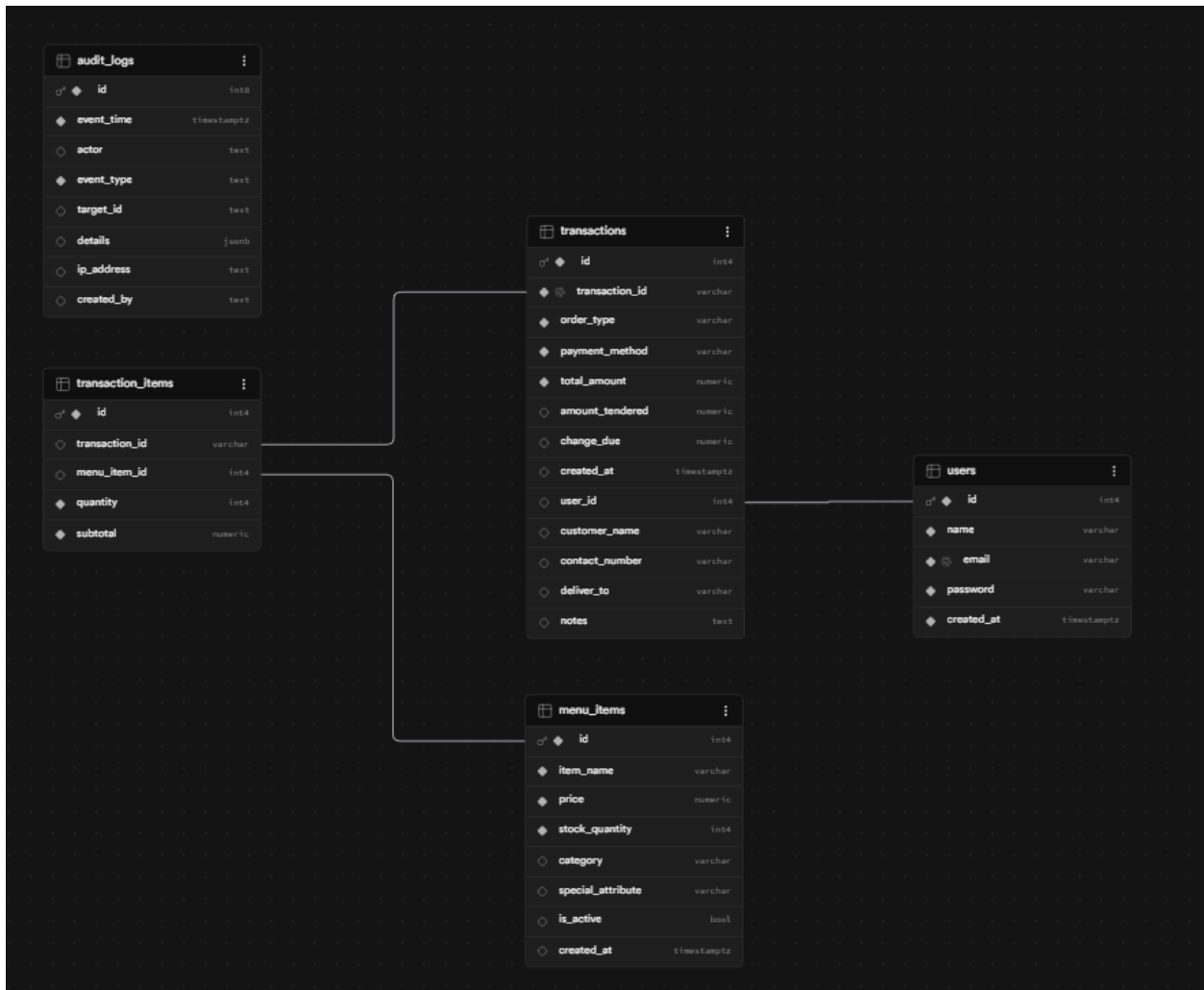


Figure 5.3 : Database Schema

3. TECH STACK

Backend

- Language: Java 17+
- Framework: Spring Boot (REST API)
- Database: Supabase (PostgreSQL)
- Driver: JDBC PostgreSQL 42.6.0

- Deployment: Vercel (Frontend) & Render (Java Backend)

Frontend

- Framework: React + Vite
- Language: TypeScript
- Styling: Tailwind CSS + Radix UI + Custom CSS
- Animations: Motion (Framer Motion)
- Deployment: Vercel (Frontend)